

A Plain-Language Guide to Stochastic Differential Equations and the Hull–White Model

For the p3 rates challenge: calibrating Hull–White to price a Bermudan swaption

No prior stochastic-calculus background assumed — we build everything from scratch.

Part 0 — The 30-second summary

Interest rates wiggle randomly over time. To price anything that depends on future rates (like a swaption), we need a **mathematical model of how the rate wiggles**. The Hull–White model is one such model. It says:

The short interest rate drifts toward a moving target, is pulled back when it strays (mean reversion), and gets kicked around by random noise each instant.

Written as an equation:

$$dr(t) = \underbrace{[\theta(t) - a r(t)] dt}_{\text{drift}} + \underbrace{\sigma(t) dW(t)}_{\text{randomness}}$$

Everything below explains what every symbol means, *why* it is built this way, and how it lets you price the Bermudan in your task.

1 What is a Stochastic Differential Equation (SDE)?

1.1 Start with an ordinary differential equation (ODE)

You already know equations like $dx/dt = 5$, meaning “ x grows at 5 units per second.” This is **deterministic**: if you know where you start, you know *exactly* where you will be at every future time, $x(t) = x(0) + 5t$. No surprises. We often write the same thing as $dx = 5 dt$: “in a tiny time step dt , x changes by $5 dt$.” It is just the ODE rearranged.

1.2 The problem: the real world is noisy

Interest rates do not move smoothly and predictably — they jiggle. A purely deterministic equation can never capture a rate that is partly random. We need to **add a random term**. That is the entire idea of an SDE:

$$dx = (\text{predictable part}) dt + (\text{random part}) \times (\text{random noise})$$

- The **predictable part** is the **drift** — the average direction of motion, same role as the right-hand side of an ODE.
- The **random part** is the **diffusion** (or volatility) — it controls *how big* the random jiggles are.
- The **random noise** itself is the new ingredient: Brownian motion.

1.3 Brownian motion $W(t)$ — the engine of randomness

Brownian motion (a *Wiener process*, hence W) is the standard model of “pure random wiggling.” Picture a speck of dust bumped by water molecules. Its key properties:

1. **Starts at zero:** $W(0) = 0$.
2. **Random increments:** over a step Δt , the change $\Delta W = W(t + \Delta t) - W(t)$ is drawn from a **normal (bell-curve)** distribution with mean 0 and variance Δt . In symbols $\Delta W \sim \mathcal{N}(0, \Delta t)$, i.e. $\Delta W = \sqrt{\Delta t} Z$ where Z is a standard normal random number.
3. **Independent increments:** what happens in one interval is independent of the past. No memory.

The crucial, slightly weird fact: because the standard deviation of a step is $\sqrt{\Delta t}$, randomness shrinks *slower* than drift as $\Delta t \rightarrow 0$. Over a tiny step, drift contributes $\sim \Delta t$ (tiny) but the random part contributes $\sim \sqrt{\Delta t}$ (bigger). **Over short horizons randomness dominates; over long horizons drift dominates.** That is why a single day of rates looks like pure noise but a decade shows a trend. In differential notation we write $dW(t)$: an infinitesimal random kick, normal with mean 0 and variance dt .

1.4 Putting it together — a general SDE

$$dx(t) = \mu(x, t) dt + \sigma(x, t) dW(t)$$

where $\mu(x, t)$ is the **drift**, $\sigma(x, t)$ the **diffusion/volatility**, and $dW(t)$ the Brownian kick. **How to read it:** “Over the next instant dt , x moves by μdt (the predictable nudge) plus σ times a random normal kick of size $\sqrt{dt}$.” A simulatable version — literally how you would code it (the **Euler–Maruyama** scheme):

```
x_next = x + mu(x, t)*dt + sigma(x, t)*sqrt(dt)*np.random.randn()
```

That one line is the computational heart of every Monte Carlo rate simulation, including Longstaff–Schwartz in your Task 3.

1.5 A word on Itô calculus (just enough)

Ordinary calculus breaks slightly when dW is involved, because dW is so jagged that $(dW)^2$ is *not* negligible — it behaves like dt . The rulebook for calculus with dW is **Itô calculus**, and its one rule to remember is

$$(dW)^2 = dt, \quad dt dW = 0, \quad (dt)^2 = 0.$$

You rarely need to derive things by hand here — but this is *why* Hull–White has clean closed-form bond prices: the math works out to Gaussian (normal) distributions, which integrate nicely.

2 Modeling interest rates: the “short rate”

2.1 What is the short rate $r(t)$?

The **short rate** $r(t)$ is the interest rate for borrowing/lending over an *infinitesimally short* period starting at time t — the “instantaneous” risk-free rate at each moment. It is useful because if you know how $r(t)$ behaves over all future times, you can price *any* interest-rate product by discounting along random paths. The price today of a **zero-coupon bond** paying \$1 at time T is the expected discount factor:

$$P(0, T) = \mathbb{E} \left[\exp \left(- \int_0^T r(s) ds \right) \right].$$

This expectation is over all random paths the short rate could take. **A short-rate model is just a specific SDE for $r(t)$** that makes this expectation computable.

2.2 What we want from a good short-rate model

1. **Mean reversion** — rates get pulled back toward a long-run level rather than wandering to infinity (a plain Brownian motion would drift away unrealistically).
2. **Fits today’s yield curve exactly** — the model must reproduce every rate the market quotes today, or every price is off from the start.
3. **Analytical tractability** — closed-form bond and European-option prices, so calibration and risk are fast (180-second budget, hundreds of repricings for Greeks).

Hull–White is the simplest model delivering all three — hence the industry workhorse, and why the task uses it.

3 The Hull–White model, term by term

$$dr(t) = [\theta(t) - ar(t)] dt + \sigma(t) dW(t)$$

3.1 The mean-reversion term: $-ar(t)$

This is the heart of the model. a is the **mean-reversion speed** (a positive constant you calibrate). Look at the sign:

- If $r(t)$ is **high**, then $-ar(t)$ is a large negative number — drift pushes the rate **down**.
- If $r(t)$ is **low** (or negative), $-ar(t)$ is positive — drift pushes the rate **up**.

So the rate is always pulled toward a central level, like a spring. Bigger a means a stronger, faster pull. **Intuition:** the half-life of a shock is $\ln 2/a$. With $a = 0.03$ a shock takes $\approx 0.69/0.03 \approx 23$ years to half-decay (slow, persistent); with $a = 0.3$ the half-life is ≈ 2.3 years (fast reversion).

3.2 The drift target: $\theta(t)$

$\theta(t)$ is a **time-dependent function**, not a constant. Combined with mean reversion, the rate reverts toward the level $\theta(t)/a$ at each instant. The clever part that makes Hull–White special: $\theta(t)$ is **not free** — it is **computed analytically so the model reproduces today’s yield curve exactly**. You do not calibrate it. The known formula is

$$\theta(t) = \frac{\partial f(0, t)}{\partial t} + a f(0, t) + \frac{\sigma^2}{2a} (1 - e^{-2at}),$$

where $f(0, t)$ is today’s **instantaneous forward rate** curve (which you build in Task 1). The takeaway: $\theta(t)$ is a plug that bends the model’s average path through every market-quoted rate — guaranteeing property 2. In practice it gets absorbed into the bond-price formula (Section 4), so you never compute it explicitly.

3.3 The volatility term: $\sigma(t) dW(t)$

$\sigma(t)$ is the **volatility** — how hard the random kicks hit. In your task it is **piecewise constant**: a different constant on each interval between exercise dates (σ_1 on $[0, T_1]$, σ_2 on $(T_1, T_2]$, etc.). This, together with a , is what you **calibrate**. Why piecewise constant? Because it gives just enough flexibility to match the **term structure of volatility** — the market quotes different implied vols for different expiries, and a single constant σ cannot fit them all. Each σ_i is a knob for its time bucket.

- Bigger $\sigma \Rightarrow$ wider spread of future rates $\Rightarrow$ more valuable options.
- σ is what swaption prices are most sensitive to $\Rightarrow$ that is your **Vega**.

3.4 Why “one-factor” and “Gaussian”?

- **One-factor**: a single source of randomness — one dW . The entire yield curve is driven by the one variable $r(t)$. A simplification (real curves can twist in more ways), but fast and adequate for this Bermudan.
- **Gaussian**: because the equation is *linear* in r (no r^2 , no $\sqrt{r}$), the solution $r(t)$ is **normally distributed**. This is the magic: normal distributions integrate in closed form (closed-form bond prices), it is why **Jamshidian’s decomposition** works, and the downside is that rates can go **negative** (the normal has no floor). In today’s low/negative-rate world that is often acceptable; it matches the Normal/Bachelier vol convention your quotes use.

4 From the SDE to actual prices (why HW is tractable)

4.1 The closed-form zero-coupon bond price

Because $r(t)$ is Gaussian, the bond-price expectation from Section 2.1 solves exactly. Under Hull–White, the price at time t (when the short rate is r) of a \$1 zero-coupon bond maturing at T is

$$P(t, T) = A(t, T) \exp(-B(t, T) r), \quad B(t, T) = \frac{1 - e^{-a(T-t)}}{a},$$

and $A(t, T)$ is a deterministic function built from today’s discount curve and a, σ (it contains the θ plug, so you never compute θ directly). **Why it matters**: this single formula is the workhorse.

Every node of your trinomial tree, or every Monte Carlo path, evaluates bond prices with it. The swap value at exercise is a sum of these bond prices — fast and exact, no nested simulation.

4.2 European swaptions: Jamshidian's decomposition

A swaption is an option on a **swap**, and a swap is a bundle of cashflows, i.e. a bundle of zero-coupon bonds (a **coupon bond**). An option on a bundle is normally hard. **Jamshidian's trick** exploits the one-factor Gaussian structure: because *every* bond price $P(t, T)$ is a **monotonic** function of the single state variable r (higher $r \Rightarrow$ lower bond prices, always), there is exactly one critical rate r^* at which the swap is exactly at-the-money. This lets you **decompose** the swaption into a **portfolio of options on individual zero-coupon bonds**, each with a **closed-form** Hull–White price (a Black-76-like formula). Sum them $\Rightarrow$ the European swaption price, analytically. This is the method for **Task 2 (calibration)**: you price European swaptions thousands of times searching for $(a, \sigma_1.. \sigma_N)$ that match market vols — closed-form Jamshidian makes each evaluation microseconds instead of a simulation.

4.3 Bermudan swaptions: why we need a tree or Monte Carlo

A **Bermudan** swaption can be exercised at *several* dates (but only once). At each exercise date you decide

$$V = \max(\text{exercise value}, \text{continuation value}),$$

where the exercise value is the value of entering the swap now (closed-form, from the bond formula) and the continuation value is the value of keeping the option alive (it depends on all later decisions). This is an **optimal stopping problem** — no single closed form — so you solve it **backward** numerically:

- **Trinomial tree**: discretize $r(t)$ onto a recombining lattice (up / stay / down, with the three branches calibrated to HW's mean and variance). Roll backward from the last date: at each node take $\max(\text{exercise}, \text{continuation})$, discounting expected values back one step. The Gaussian structure is what makes the tree *recombine* (up-then-down lands on the same node as down-then-up), keeping it small.
- **Monte Carlo + Longstaff–Schwartz**: simulate many random $r(t)$ paths (the Euler step from Section 1.4). The trouble with MC and early exercise is you do not know the continuation value along a path; Longstaff–Schwartz estimates it by **regressing** realized future payoffs against functions of the current state, at each exercise date, working backward. The fitted value *is* the continuation-value estimate.

Either way, the **early-exercise premium** (the extra value of a Bermudan over a European) falls out of this backward induction.

4.4 Greeks: bump-and-reprice (Task 4)

- **Vega** = sensitivity to each market vol quote: bump one vol by +1bp, **re-calibrate** (a, σ) , re-price, take the difference.
- **Delta** = sensitivity to each zero rate: bump one zero rate by +1bp, rebuild the curve, re-calibrate, re-price, difference.

This is why **speed matters**: every Greek is a full re-calibration plus re-pricing. Closed-form Jamshidian (calibration) plus an efficient tree (pricing) keep you inside the 180-second budget.

5 Jamshidian's decomposition — step by step

The problem. A payer swaption is the right to enter a swap, and a swap is worth a *bundle* of cashflows (a coupon bond). An option on a *sum* of things is normally hard, because the sum's value depends on how all the pieces move together.

Jamshidian's insight (works precisely because Hull–White is one-factor): everything at expiry depends on a single random number — the state x . And every bond price $P(\text{expiry}, T \mid x)$ is **monotonic** in x (higher $x \Rightarrow$ higher rates $\Rightarrow$ lower bond price). So the swap's value is also monotonic in x : there is **exactly one** critical state x^* where the swap is precisely at-the-money (worth zero). On one side of x^* you exercise, on the other you do not. Because that boundary is a single point, the option on the bundle **splits into a sum of independent options on each individual bond**, each struck at that bond's value at x^* — and each of those has a closed-form (Black-style) price.

The recipe (exactly what your EuroPricer does):

1. Write the swap value at expiry as a function of the state x :

$$\text{swap}(x) = \underbrace{1}_{\text{receive floating}} - \underbrace{\sum_i K \tau_i P(e, T_i \mid x)}_{\text{pay fixed leg}} - \underbrace{P(e, T_n \mid x)}_{\text{final notional}},$$

which is positive when the payer is in-the-money.

2. Find the critical state x^* where $\text{swap}(x^*) = 0$. This is a one-dimensional root-find (your code uses `brentq` over ± 8 standard deviations, with guards for the all-in / all-out cases).
3. Turn each cashflow into a bond option. At x^* , each bond price $P(e, T_i \mid x^*)$ becomes a **strike** K_i . The swaption is now a **portfolio of bond put options**, one per cashflow, each weighted by its cashflow size $c_i = K \tau_i$ (+1 at the final date).
4. Price each bond put in closed form with a Black-76-style formula using the bond's Hull–White volatility $v_i = B(e, T_i) \cdot \text{std}_e$, then **sum**: $\text{price} = \sum_i c_i \cdot \text{Put}_i$.

Why a put, for a payer? A payer gains when rates *rise*, i.e. when bond prices *fall*. An option that pays off when a bond price falls below a strike is a **put** on that bond. So a payer swaption equals a basket of bond puts. (A receiver would be a basket of bond calls.)

Payoff: an exact European price in microseconds, with no simulation — which is what makes the thousands of repricings in calibration (Task 2) feasible inside the time limit.

6 The backward-induction engine (your Bermudan PDE pricer)

A Bermudan can be exercised at several dates, and the value today depends on a whole chain of future decisions — so you cannot price it forward. You solve it **backward**, from the last date to today. Your code does this with a **PDE on a grid**, which is a smooth, deterministic cousin of a tree.

6.1 The state variable

Instead of tracking r directly, work with $x(t) = r(t) - \alpha(t)$, where $\alpha(t)$ is the deterministic (curve-fitting) part. x is a clean mean-reverting process centred at 0 (an **Ornstein–Uhlenbeck** process), which keeps the grid symmetric and stable. The short rate is recovered as $r = x + \alpha(t)$.

6.2 The grid

- **Space:** a row of possible x values from $-x_{\max}$ to $+x_{\max}$ (your code: 401 points, $x_{\max} \approx 6$ standard deviations — wide enough to hold essentially all the probability).
- **Time:** small steps (daily, $1/365$) from today out to `swap_end`, with the exercise dates inserted *exactly* so decisions land on grid times.

6.3 The backward march

1. **Start at the end.** At `swap_end` the option is worth 0 everywhere (terminal condition $V = 0$).
2. **Step back one dt at a time.** Each step answers: “given the value one step in the future, what is it worth now?” — the discounted expected future value, accounting for (a) the random spreading of x (diffusion $\frac{1}{2}\sigma^2 V_{xx}$), (b) the mean-reverting drift ($-a x V_x$), and (c) discounting at the current rate ($-(x + \alpha) V$). Your code does this with one **implicit finite-difference solve** per step — a fast tridiagonal system (`solve_banded`). “Implicit” just means a stable scheme that will not blow up even with large steps.
3. **At every exercise date, apply the early-exercise test.** Compute the value of exercising now (entering the payer swap at that node’s rate):

$$\text{exercise}(x) = 1 - P(t, \text{swap_end} \mid x) - K \cdot \text{annuity}(x),$$

then take, at every grid point,

$$V = \max(\text{continuation}(x), \text{exercise}(x)).$$

You exercise wherever exercising beats holding — this single max is what captures the **early-exercise premium**.

4. **Keep marching** until $t = 0$.

6.4 Read off the answer

At $t = 0$ today’s state is $x = 0$, so the price is V interpolated at $x = 0$, multiplied by the notional.

6.5 Why backward, and how it relates to trees / Longstaff–Schwartz

You must go backward because the exercise decision at each date needs the value of *continuing*, which depends on all *later* dates. Starting at the end and working back means the continuation value is already known when you reach each decision — the essence of backward induction (and of optimal-stopping problems generally). Same idea, different machinery: a **tree** discretizes into up/down branches; **Longstaff–Schwartz** uses simulated paths plus regression to estimate continuation values; your **PDE** discretizes the governing equation on a fixed grid and solves it directly. For a clean one-factor model like this, the PDE is smooth, stable, and very accurate.

7 The whole pipeline in one picture

Task 1	Task 2	Task 3	Task 4
Zero curve -> DFs, forwards	Calibrate a, sigma: choose params so HW European prices (Jamshidian) match market vols (min RMSE)	Price Bermudan: backward induct on tree / LSMC, max(exercise, continuation)	Bump & reprice -> Delta, Vega
curve feeds theta	the SDE gets its a and sigma pinned	the SDE is simulated/lattice'd	sensitivities of the price

8 Glossary of every symbol

Symbol	Name	What it is	Free or fixed?
$r(t)$	short rate	instantaneous risk-free rate at t	the random variable
$dr(t)$	change in r	how r moves over an instant dt	—
a	mean-reversion speed	how strongly r is pulled back	calibrated
$\theta(t)$	drift target	bends model to fit today's curve	fixed (from curve)
$\sigma(t)$	volatility	size of random kicks, piecewise-constant	calibrated
$dW(t)$	Brownian kick	random normal noise, variance dt	the randomness
$W(t)$	Brownian motion	cumulative random walk	—
$P(t, T)$	bond price	value at t of \$1 paid at T	closed-form
$f(0, t)$	forward rate	today's instantaneous forward curve	from Task 1
A, B	HW coefficients	building blocks of $P(t, T)$	from a, σ , curve

9 Common pitfalls & sanity checks

1. **Number of σ parameters is dynamic.** It equals the number of breakpoint intervals $[0, T_1, \dots, T_k, \text{swap_end}]$. Read it from the input every time — never hardcode (hidden cases differ).
2. **Negative rates are allowed** in Hull–White (Gaussian). Do not floor them at zero — the Normal/Bachelier convention expects them.
3. $\theta(t)$ is **never calibrated**. If you are fitting θ , you have misunderstood — it is analytic from the curve. Only a and σ are free.
4. **Mind the metric.** Calibration minimizes RMSE of vols in bps, but pricing is scored on MAPE of the dollar price — different objectives.
5. **Recombining tree:** verify up-then-down lands on down-then-up, or the tree blows up exponentially and busts the time limit.

6. **Sanity-check the bond formula:** $P(t, t) = 1$, and $P(0, T)$ must match your Task-1 discount factors exactly. If not, $A(t, T)$ is wrong.
7. **Greek signs:** a *payer* swaption gains value when rates rise (positive delta to rates); vega is positive everywhere (more vol = more option value).

10 If you want to go deeper

- **Brigo & Mercurio, *Interest Rate Models — Theory and Practice*:** the definitive reference; Chapter 3 covers Hull–White and Jamshidian in full.
- **Hull, *Options, Futures, and Other Derivatives*:** gentler, with the trinomial tree construction step by step.
- **Original papers:** Hull & White (1990) for the model; Jamshidian (1989) for the decomposition; Longstaff & Schwartz (2001) for the MC regression method.

The one sentence to remember: Hull–White is just an SDE that makes the short rate spring back toward a curve-fitting target while getting randomly kicked — and because it is linear (hence Gaussian), it hands you closed-form bond and European prices, which you then stack into a backward-induction pricer for the Bermudan.